SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1419 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:100**
Mode: In Class **Hours : 60 mts**

Annexure A

EXPERIMENTS

I Akash B (192511257) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Akash B

Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 20**	**/ 25**	**/ 20**	**/ 15**	**/ 100**

Annexure A

EXPERIMENTS

Q1.
You are designing a compiler module that evaluates arithmetic expressions using syntax-directed definitions (SDD). The system must:
(i) Accept arithmetic expressions with operators +, -, *, / and parentheses
(ii) Construct a syntax tree and evaluate expressions using synthesized attributes
(iii) Respect operator precedence and associativity rules
(iv) Produce the final computed value of the expression
Demonstrate how SDD rules are applied for evaluating expressions and explain the bottom-up evaluation process using annotated parse trees.

Aim:

To design and implement a compiler module that evaluates arithmetic expressions using **Syntax-Directed Definitions (SDD)** with synthesized attributes, while respecting **operator precedence, associativity, and parentheses**.

Requirements:

The system should:

1. Accept arithmetic expressions containing +, -, *, /, and parentheses.
2. Construct a syntax tree for the expression.
3. Evaluate the expression using **synthesized attributes**.
4. Follow normal arithmetic precedence:
    - Parentheses ()
    - Multiplication and division * /
    - Addition and subtraction + -
5. Follow left associativity for +, -, *, and /.
6. Produce the final numerical value.

Grammar Used:

$E \rightarrow E + T$

$E \rightarrow E - T$

$E \rightarrow T$

$T \rightarrow T * F$

T → T / F

T → F

F → (E)

F → num

- E = Expression
- T = Term
- F = Factor
- num = Number

This grammar automatically gives:

- * and / higher precedence than + and -
- +, -, *, / left associativity
- Parentheses the highest priority.

Syntax-Directed Definition Rules

The synthesized attribute val stores the value of each expression.

Production	**Semantic Rule**
E → E1 + T	E.val = E1.val + T.val
E → E1 - T	E.val = E1.val - T.val
E → T	E.val = T.val
T → T1 * F	T.val = T1.val * F.val
T → T1 / F	T.val = T1.val / F.val
T → F	T.val = F.val
F → (E)	F.val = E.val
F → num	F.val = num.lexval

Here lexval represents the numerical value of the token.

Example Expression

3 + 4 * 2 - (6 / 3)

Because multiplication and division have higher precedence:

3 + (4 * 2) - (6 / 3)

3 + 8 - 2

= 11 - 2= 9

Annotated Parse Tree

E [val = 9]

/ | \

E [val=11] - T [val=2]

/ | \

E[val=3] + T[val=8]

| / | \

T[val=3] T[val=4] * F[val=2]

| | |

F[val=3] F[val=4] num[2]

|

num[3]

For (6 / 3):

F [val=2]

|

(E)

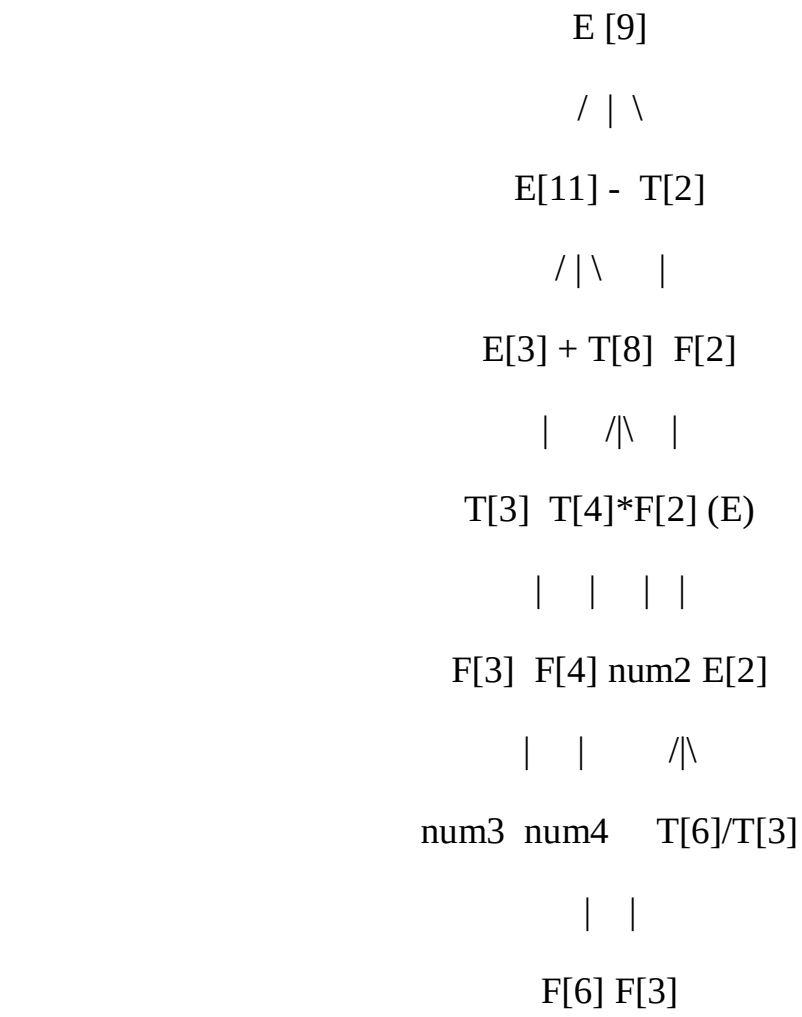
|

E [val=2]

/ | \

T[6] / T[3]

Bottom-Up Evaluation Diagram:

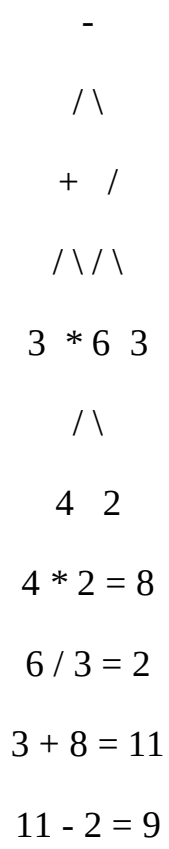


Evaluation order:

3 → 4 → 2 → 4×2=8 → 6 → 3 → 6÷3=2

→ 3+8=11 → 11−2=9

Syntax Tree



4 * 2 = 8

6 / 3 = 2

3 + 8 = 11

11 - 2 = 9

Why Synthesized Attributes Are Used

A synthesized attribute is calculated using information from the children of a node.

E → E1 + T

E.val = E1.val + T.val

The parent E.val depends only on the child values E1.val and T.val.

Parent

↑

|

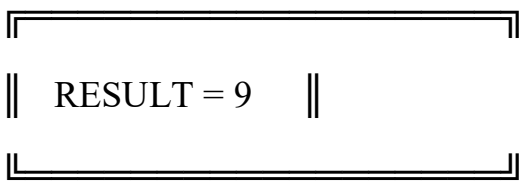
Children

This makes the SDD suitable for **bottom-up parsing and evaluation**.

Conclusion

The arithmetic expression evaluator can be implemented using an SDD with synthesized attributes. The grammar separates expressions, terms, and factors to enforce precedence and left associativity. During bottom-up evaluation, numeric values are first synthesized at the leaves and then propagated upward through the parse tree. For the expression:

3 + 4 * 2 - 6 / 3



Thus, SDD provides a systematic way to **construct the parse/syntax tree and evaluate the expression using bottom-up synthesized attributes**.

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>

char exp[100];

int pos = 0;

typedef struct Node {

    char op;

    int value;

    struct Node *left, *right;

} Node;

Node* numberNode(int value) {

    Node *n = malloc(sizeof(Node));
```

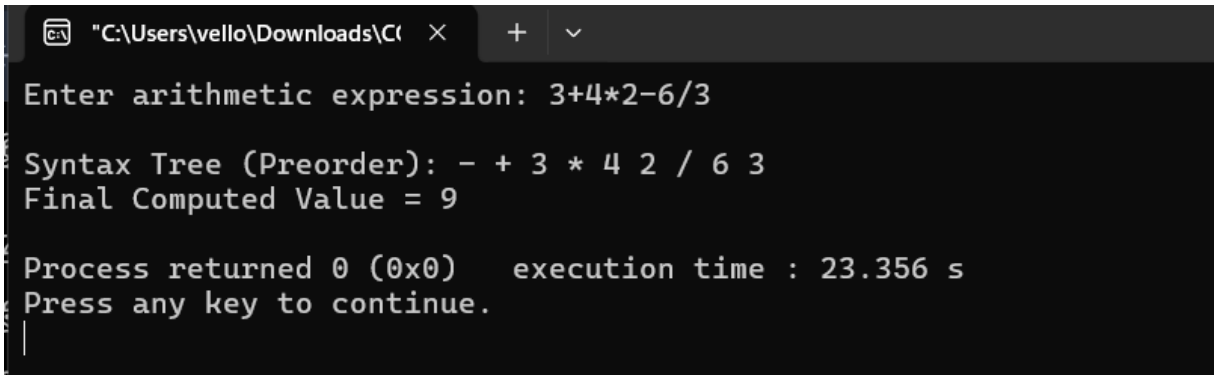
```
    n->op = '\0';
    n->value = value;
    n->left = n->right = NULL;
    return n;
}
Node* operatorNode(char op, Node *left, Node *right) {
    Node *n = malloc(sizeof(Node));
    n->op = op;
    n->left = left;
    n->right = right;
    return n;
}
Node* expression();
Node* factor() {
    while (exp[pos] == ' ')
        pos++;
    if (exp[pos] == '(') {
        pos++;
        Node *n = expression();
        pos++;
        return n;
    }
    int num = 0;
```

```
    while (isdigit(exp[pos])) {
        num = num * 10 + (exp[pos] - '0');
        pos++;
    }
    return numberNode(num);
}
Node* term() {
    Node *left = factor();
    while (exp[pos] == '*' || exp[pos] == '/') {
        char op = exp[pos++];
        Node *right = factor();
        left = operatorNode(op, left, right);
    }
    return left;
}
Node* expression() {
    Node *left = term();
    while (exp[pos] == '+' || exp[pos] == '-') {
        char op = exp[pos++];
        Node *right = term();
        left = operatorNode(op, left, right);
    }
    return left;
```

```
}
/* SDD synthesized attribute evaluation */
int evaluate(Node *root) {
    if (root->op == '\0')
        return root->value;
    int left = evaluate(root->left);
    int right = evaluate(root->right);
    switch (root->op) {
        case '+':
            return left + right;
        case '-':
            return left - right;
        case '*':
            return left * right;
        case '/':
            if (right == 0) {
                printf("Error: Division by zero!\n");
                exit(1);
            }
            return left / right;
    }
    return 0;
}
```

```
/* Display syntax tree */
void preorder(Node *root) {
    if (root == NULL)
        return;
    if (root->op == '\0')
        printf("%d ", root->value);
    else
        printf("%c ", root->op);
    preorder(root->left);
    preorder(root->right);
}
int main() {
    Node *root;
    int result;
    printf("Enter arithmetic expression: ");
    fgets(exp, sizeof(exp), stdin);
    root = expression();
    printf("\nSyntax Tree (Preorder): ");
    preorder(root);
    result = evaluate(root);
    printf("\nFinal Computed Value = %d\n", result);
    return 0;
}
```

Output:



Q2.
In a compiler, intermediate code generation often uses postfix expressions for efficient evaluation. The system must:
(i) Accept a postfix expression as input
(ii) Use a stack-based approach to evaluate the expression
(iii) Demonstrate bottom-up computation of intermediate results
(iv) Display the final evaluated result
Write a C program to evaluate postfix expressions and explain how this process relates to S-attributed definitions in compilers.

Objective:

To evaluate a postfix expression using a **stack-based approach** and demonstrate how the evaluation process relates to **S-attributed definitions** in compiler design.

Concepts Used

- Postfix expression
- Stack data structure
- Bottom-up evaluation
- Intermediate results
- Operators and operands
- Synthesized attributes
- S-attributed definitions

S-Attributed Definition

An S-attributed definition uses only **synthesized attributes**. The value of a parent node is calculated from the values of its child nodes.

Example:

E → E1 + E2

E.val = E1.val + E2.val

E → E1 - E2

E.val = E1.val - E2.val

E → E1 * E2

E.val = E1.val * E2.val

E → E1 / E2

E.val = E1.val / E2.val

The postfix evaluation also follows a **bottom-up computation**, because intermediate values are calculated before the final result.

Experimental Design:

Algorithm

1. Start the program.
2. Read the postfix expression.
3. Initialize an empty stack.
4. Scan the expression from left to right.
5. If the symbol is an operand, push it onto the stack.
6. If the symbol is an operator:
    - Pop the second operand.
    - Pop the first operand.
    - Perform the operation.
    - Push the result onto the stack.
7. Continue until the expression ends.
8. Pop the final value from the stack.
9. Display the result.
10. Stop.

Analysis of Results:

Input:

23+45-*

(2 + 3) * (4 - 5)

Step	Symbol	Operation	Stack
1	2	Push 2	2
2	3	Push 3	2 3
3	+	2 + 3 = 5	5
4	4	Push 4	5 4
5	5	Push 5	5 4 5
6	-	4 - 5 = -1	5 -1
7	*	5 × -1 = -5	-5

Output:

Enter postfix expression: 23+45-*

Final Result = -5

Bottom-Up Computation

2 + 3 = 5

↓

4 - 5 = -1

↓

5 × (-1) = -5

The final result is:

-5

The experiment successfully demonstrates that postfix expressions can be evaluated efficiently using a stack. Each intermediate result is computed from previously evaluated operands, similar to the computation of **synthesized attributes in an S-attributed definition**.

Code:

```
#include <stdio.h>
#include <ctype.h>
#define MAX 100
int stack[MAX];
int top = -1;
void push(int value)
{
    stack[++top] = value;
}
int pop()
{
    return stack[top--];
}
int main()
{
    char postfix[MAX];
    int i, a, b, result;
    printf("POSTFIX EXPRESSION EVALUATION\n");
    printf("----------------------------\n");
    printf("Enter postfix expression: ");
    scanf("%s", postfix);
```

```
for (i = 0; postfix[i] != '\0'; i++)
{
    if (isdigit(postfix[i]))
    {
        push(postfix[i] - '0');
    }
    else
    {
        b = pop();
        a = pop();
        switch (postfix[i])
        {
            case '+':
                result = a + b;
                printf("%d + %d = %d\n", a, b, result);
                break;
            case '-':
                result = a - b;
                printf("%d - %d = %d\n", a, b, result);
                break;
            case '*':
                result = a * b;
```

```
                printf("%d * %d = %d\n", a, b, result);
                break;
            case '/':
                if (b == 0)
                {
                    printf("Division by zero is not allowed.\n");
                    return 1;
                }
                result = a / b;
                printf("%d / %d = %d\n", a, b, result);
                break;
            default:
                printf("Invalid operator\n");
                return 1;
        }
        push(result);
    }
  }
  result = pop();
  printf("----------------------------\n");
  printf("Final Result = %d\n", result);
  return 0;
}
```

Output:

```
POSTFIX EXPRESSION EVALUATION
-----------------------------
Enter postfix expression: 23+45-*
2 + 3 = 5
4 - 5 = -1
5 * -1 = -5
-----------------------------
Final Result = -5

Process returned 0 (0x0)   execution time : 18.045 s
Press any key to continue.
```

Result:

Thus, the postfix expression was successfully evaluated using a stack-based approach. The experiment demonstrates bottom-up computation of intermediate results and its relationship with S-attributed definitions in compiler design.

Q3.
You are developing a compiler component that uses L-attributed definitions for expression evaluation. The system must:
(i) Accept arithmetic expressions such as a + b * c
(ii) Use inherited attributes to pass information during parsing
(iii) Ensure correct operator precedence
(iv) Perform evaluation using a top-down approach
Write a C program that simulates L-attributed evaluation and explain how inherited attributes are used to propagate values during parsing.

Aim

To write a C program that simulates **L-attributed evaluation** of an arithmetic expression using inherited attributes, while maintaining correct operator precedence and performing evaluation in a **top-down manner**.

Grammar

The following grammar is used:

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

T' → * F T' | ε

F → number

The grammar ensures that * has higher precedence than +.

Code:

```
#include <stdio.h>
#include <ctype.h>
char expr[100];
int pos = 0;
/* Function prototypes */
int E();
int Eprime(int inherited);
int T();
int Tprime(int inherited);
int F();
/* E -> T E' */
int E()
{
    int value;
    value = T();
    /* Pass T's value as inherited attribute to E' */
    return Eprime(value);
}
/* E' -> + T E' | epsilon */
```

```
int Eprime(int inherited)
{
    int value;
    if (expr[pos] == '+')
    {
        pos++;
        value = T();
        /* Pass updated value as inherited attribute */
        return Eprime(inherited + value);
    }
    return inherited;
}
/* T -> F T' */
int T()
{
    int value;
    value = F();
    /* Pass F's value as inherited attribute to T' */
    return Tprime(value);
}
/* T' -> * F T' | epsilon */
int Tprime(int inherited)
{
```

```
    int value;
    if (expr[pos] == '*')
    {
        pos++;
        value = F();
        /* Pass updated value as inherited attribute */
        return Tprime(inherited * value);
    }
    return inherited;
}
/* F -> number */
int F()
{
    int value = 0;
    while (isdigit(expr[pos]))
    {
        value = value * 10 + (expr[pos] - '0');
        pos++;
    }
    return value;
}
int main()
{
```

```
    int result;
    printf("L-ATTRIBUTED EXPRESSION EVALUATION\n");
    printf("---------------------------------\n");
    printf("Enter expression: ");
    scanf("%s", expr);
    result = E();
    printf("---------------------------------\n");
    printf("Expression = %s\n", expr);
    printf("Result = %d\n", result);
    return 0;
}
```

Output:

```
L-ATTRIBUTED EXPRESSION EVALUATION
---------------------------------
Enter expression: 2+3*4
---------------------------------
Expression = 2+3*4
Result = 14

Process returned 0 (0x0)   execution time : 12.820 s
Press any key to continue.
```

Q4.
A compiler's semantic analyzer must check type compatibility between expressions. The system should:
(i) Accept two type expressions such as int, float, int*, char*
(ii) Compare the types for equivalence
(iii) Handle both basic and pointer types
(iv) Display whether the types are equivalent or not
Write a C program to perform type equivalence checking and explain how compilers use this mechanism during semantic analysis.

Aim

To write a C program that checks whether two type expressions are **equivalent or not**, including basic types such as int, float, char and pointer types such as int* and char*.

Algorithm

1. Start the program.
2. Read the first type expression.
3. Read the second type expression.
4. Compare the two type expressions.
5. If both are basic types, compare their names.
6. If both are pointer types:
    - Remove *.
    - Compare the base types.
7. Display whether the types are equivalent or not.
8. Stop.

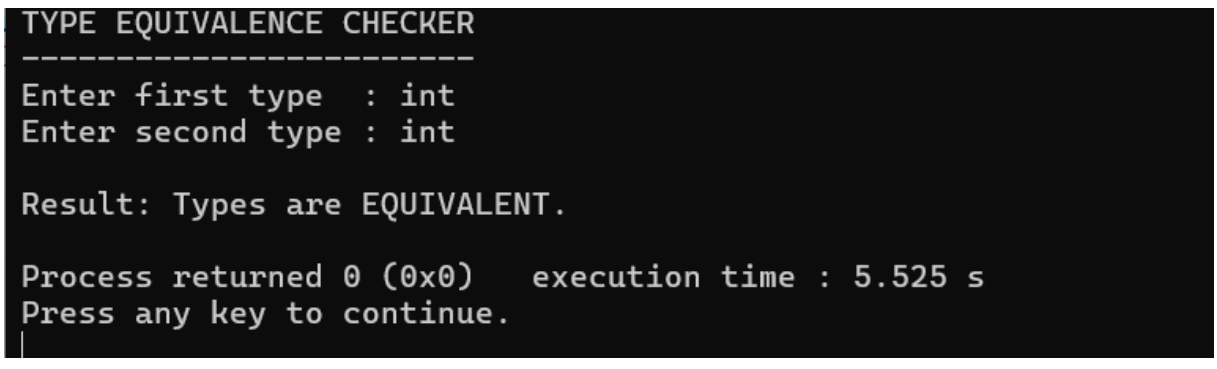
Code:

```
#include <stdio.h>

#include <string.h>

int isBasicType(char type[])

{

   if (strcmp(type, "int") == 0 ||

      strcmp(type, "float") == 0 ||

      strcmp(type, "char") == 0)

      return 1;

   return 0;

}

int isPointerType(char type[])

{
```

```
    int len = strlen(type);
    if (len > 1 && type[len - 1] == '*')
        return 1;
    return 0;
}
int equivalent(char type1[], char type2[])
{
    /* Basic type comparison */
    if (isBasicType(type1) && isBasicType(type2))
    {
        return strcmp(type1, type2) == 0;
    }
    /* Pointer type comparison */
    if (isPointerType(type1) && isPointerType(type2))
    {
        char base1[20], base2[20];
        strcpy(base1, type1);
        strcpy(base2, type2);
        base1[strlen(base1) - 1] = '\0';
        base2[strlen(base2) - 1] = '\0';
        return strcmp(base1, base2) == 0;
    }
```

```
    return 0;
}
int main()
{
    char type1[20], type2[20];
    printf("TYPE EQUIVALENCE CHECKER\n");
    printf("-----------------------\n");
    printf("Enter first type  : ");
    scanf("%s", type1);
    printf("Enter second type : ");
    scanf("%s", type2);
    if (equivalent(type1, type2))
        printf("\nResult: Types are EQUIVALENT.\n");
    else
        printf("\nResult: Types are NOT EQUIVALENT.\n");
    return 0;
}
```

Output:



Q5.
You are designing a semantic analyzer for a compiler that detects type errors in arithmetic expressions. The system must:
(i) Accept operands along with their types
(ii) Validate operations such as +, -, , /
(iii) Detect invalid combinations such as int + char
(iv) Report whether the expression is valid or contains a type error
Write a C program to perform type checking and explain how semantic analysis ensures correctness in program execution.

Aim

To write a C program that performs **type checking** for arithmetic expressions and detects invalid combinations such as int + char*.

Concepts Used

- Semantic analysis
- Type checking
- Basic data types: int, float, char
- Pointer type: int*, char*
- Arithmetic operators: +, -, *, /
- Type errors

Algorithm

1. Read the first operand and its type.
2. Read the arithmetic operator.
3. Read the second operand and its type.
4. Check whether both operands have valid arithmetic types.
5. Reject pointer types in normal arithmetic operations.
6. Check the compatibility of the two operand types.
7. Display whether the expression is valid or contains a type error.

Code:

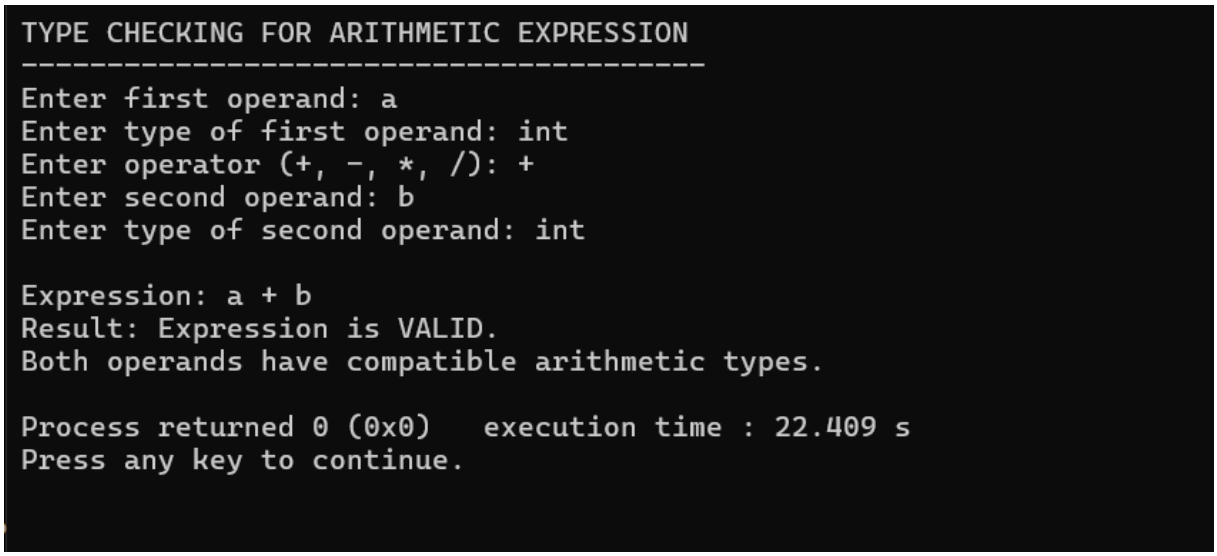
```
#include <stdio.h>

#include <string.h>

int isNumeric(char type[])
```

```
{
    if (strcmp(type, "int") == 0 ||
        strcmp(type, "float") == 0 ||
        strcmp(type, "char") == 0)
        return 1;
    return 0;
}
int isPointer(char type[])
{
    int len = strlen(type);
    if (len > 1 && type[len - 1] == '*')
        return 1;
    return 0;
}
int main()
{
    char operand1[20], operand2[20];
    char type1[20], type2[20];
    char op;
    printf("TYPE CHECKING FOR ARITHMETIC EXPRESSION\n");
    printf("---------------------------------------\n");
    printf("Enter first operand: ");
    scanf("%s", operand1);
```

```
printf("Enter type of first operand: ");
scanf("%s", type1);
printf("Enter operator (+, -, *, /): ");
scanf(" %c", &op);
printf("Enter second operand: ");
scanf("%s", operand2);
printf("Enter type of second operand: ");
scanf("%s", type2);
printf("\nExpression: %s %c %s\n",
      operand1, op, operand2);
/* Check operator */
if (op != '+' && op != '-' &&
   op != '*' && op != '/')
{
   printf("Result: Invalid operator.\n");
   return 0;
}
/* Check pointer types */
if (isPointer(type1) || isPointer(type2))
{
   printf("Result: TYPE ERROR\n");
   printf("Arithmetic operation cannot be performed "
          "with pointer type.\n");
```

```
    }
    /* Check numeric types */
    else if (isNumeric(type1) && isNumeric(type2))
    {
        printf("Result: Expression is VALID.\n");
        printf("Both operands have compatible arithmetic types.\n");
    }
    else
    {
        printf("Result: TYPE ERROR\n");
        printf("Incompatible operand types.\n");
    }
    return 0;
}
```

Output:



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation